УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 13

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ С ПРИМЕНЕНИЕМ НАСЛЕДОВАНИЯ И ПОЛИМОРФИЗМА

ЛАБОРАТОРНАЯ РАБОТА № 13.1

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ С ПРИМЕНЕНИЕМ НАСЛЕДОВАНИЯ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ с применением наследования

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Наследование

Наследование является, пожалуй, самым главным понятием ООП. Предположим, есть класс (например, **Class1**). При помощи наследования можно создать новый класс (например, **Class2**), в котором будет реализован доступ ко всем атрибутам и методам класса **Class1**:

```
class Class1:        # Базовый класс
  def  func1(self):
     print ("Метод func1() класса Class1")
  def  func2(self):
     print ("Метод func2() класса Class1")
class Class2 (Class1): # Класс Class2 наследует класс Ciass1
  def  func3(self):
     print ("Метод func3() класса Class2")
c = Class2()         # Создаем экземпляр класса Class2
c.func1()          # Выведет: Метод func1() класса Ciass1
c.func2()          # Выведет: Метод func2() класса Ciass1
c.func3()          # Выведет: Метод func3() класса Ciass1
```

Результат работы приложения:

```
Метод func1() класса Class1
Метод func2() класса Class1
Метод func3() класса Class2
```

Как видно из примера, класс **Class1** указывается внутри круглых скобок в определении класса **Class2**. Таким образом, класс **Class2** наследует все атрибуты и методы класса **Class1**. Класс **Class1** называется **базовым** или **суперклассом**, а класс **Class2** - **производным** или **подклассом**.

Если имя метода в классе **Class2** совпадает с именем метода класса **Class1**, то будет использоваться метод из класса **Class2**. Чтобы вызвать одноименный метод из базового класса, следует указать перед методом название базового класса. Кроме того, в первом параметре метода необходимо явно указать ссылку на экземпляр класса. Рассмотрим это на следующем примере:

```
class Class1:                # Базовый класс
   def  __init__(self):
      print ("Конструктор базового класса")
   def  func1(self):
      print ("Метод func1() класса Class1")
class Class2 (Class1):       # Класс Class2 наследует класс Ciass1
   def  __init__(self):
      print ("Конструктор производного класса")
      Class1.__init__(self) # Вызываем конструктор базового класса
   def  func1(self):
      print ("Метод func1() класса Class2")
      Class1.func1(self)    # Вызываем метод базового класса
c = Class2()                 # Создаем экземпляр класса Class2
c.func1()                    # Вызываем метод func1()
```

Результат работы приложения:

```
Конструктор производного класса
Конструктор базового класса
Метод func1() класса Class2
Метод func1() класса Class1
```

Чтобы вызвать одноименный метод из базового класса, можно также воспользоваться функцией **super ()**. Формат функции:

```
super([<Класс>, <Указатель self>])
```

С помощью функции **super ()** инструкцию

```
Class1.__init__(self)            #  Вызываем конструктор базового класса
```

можно записать так:

```
super().__init__()           #  Вызываем конструктор базового класса
```

или так:

```
super (Class2, self).__init__() # Вызываем конструктор базового класса
```

Обратите внимание на то, что при использовании функции **super ()** не нужно явно передавать указатель **self** в вызываемый метод. Кроме того, в первом параметре функции **super ()** указывается производный класс, а не базовый. Поиск идентификатора будет производиться во всех базовых классах. Результатом поиска станет первый найденный идентификатор в цепочке наследования.

Наследование в ООП предоставляет возможность создавать класс с полями, свойствами и методами, которые могут быть использованы в других классах. В терминах ООП базовый класс называется **наследуемым** (родительским, суперклассом), а производный от него – **наследующим** (подклассом, производным). При наследовании объект производного класса является частным случаем объекта базового класса. Например, автомобиль является частным случаем транспортного средства. В производном классе будет доступ ко всем

атрибутам и методам базового класса. Возможности базового класса при этом остаются неизменными.

Так, можно создать базовый класс **Student** и определить в нем метод **information()** (информация). Затем на основе этого класса создать два отдельных класса: **Attendance** (посещаемость) и **Perfomance** (успеваемость). Они используют ту же логику для получения информации, что и класс **Student**, поэтому могут унаследовать от него метод **information().** Это позволяет программисту один раз написать общий код, который затем будет проще поддерживать. Функциональность объектов нового класса можно расширить за счет добавления собственных атрибутов и методов.

Синтаксис производного класса следующий:

Class Имя_произвольного_класса(Имя родительского_класса):
Оператор1
Оператор2
…
ОператорN

Ниже показано создание базового класса **Human** (Человек), а также двух дочерних: **Prepod** (Преподаватель) и **Student** (Студент).

```
class Human():
    name=""
class Prepod(Human):
    kafedra="" #Название кафедры
    dolgnost="" #Должность
class Student(Human):
    fakultet="" #Название факультета
    gruppa="" #Номер группы
```

Если создадим метод в базовом классе **Human**, то он будет наследоваться в методах **Prepod** и **Student** , что и показано ниже,

```
class Human():
    name=" "
    def __init__(self): #Конструктор базового класса
        print("Создан класс Человек")
#---------------------------------------
class Prepod(Human):
    kafedra="" #Название кафедры
    dolgnost="" #Должность
#---------------------------------------
class Student(Human):
    fakultet=""#Название факультета
    gruppa=""#Номер группы
#---------------------------------------
h=Human()
pr=Prepod()
st=Student()
```

В следующем примере показано, что если дочерние классы содержат собственные методы, то они и будут выполняться при вызове соответствующих объектов.

```
class Human():
    name=""
    def __init__(self): #Конструктор базового класса
        print("Создан класс Человек")
#-------------------------------------
class Prepod(Human):
    kafedra=""  #HНазвание кафедры
    dolgnost="" #Должность
    def __init__(self): #Конструктор дочернего класса
        print("Создан класс Преподаватель")
#-------------------------------------
class Student(Human):
    fakultet="" #Название факультета
    gruppa="" #Номер группы
    def __init__(self): #Конструктор дочернего класса
        print("Создан класс студент")
#---------Создание экземплялров классов-----------------------------
b=Human()
pr=Prepod()
st=Student()
```

Для того чтобы вызвать конструктор базового класса из конструктора дочернего класса, напишем в каждом из дочерних конструкторов оператор **Human.__init__**(self), так как это показано ниже.

```
class Human():
    name=""
    def __init__(self): #Конструктор базового класса
        print("Создан класс Человек")
#-------------------------------------
class Prepod(Human):
    kafedra="" #Название кафедры
    dolgnost="" #Должность
    def __init__(self): #Конструктор дочернего класса
        Human. __init__(self)
        print("Создан класс Преподаватель")
#-------------------------------------
class Student(Human):
    fakultet="" #Название факультета
    gruppa="" #Номер группы
    def __init__(self): #Конструктор дочернего класса
        Human. __init__(self)
        print("Создан класс студент")
#-------------------------------------
```

```
h=Human()
pr=Prepod()
st=Student()
```

Три раза вызвав конструктор базового класса, в котором осуществлялся вывод строки "Создан класс Человек", столько же раз получим соответствующий вывод по результатам выполнения программы.

В следующем примере создан родительский класс **Human** (Человек). В нем реализован метод-конструктор с атрибутом **name** и метод **info.** Два дочерних класса **Prepod** и **Student** будут наследовать родительский класс и, кроме того, обладать атрибутами **kafedra** (Название кафедры), **dolgnost** (Должность) в классе **Prepod** и **fakultet** (Факультет), **gruppa** (Группа) в классе **Student**. В основной части кода метод-конструктор **def __init__** вызывается из каждого дочернего класса оператором **Human.__init__**(self,name), т. е. код родительского класса используется многократно.

```
class Human():
    def __init__(self,name):
        self.name=name
        print ("Создан класс Человек")
    def info(self):
        print(self.name,end="")
#------------------------------------------------------------
class Prepod(Human):
    def __init__ (self,name,kafedra,dolgnost):
        Human.__init__(self,name)
        self. kafedra=kafedra #Название кафедры
        self.dolgnost=dolgnost #Должность
        print ("Создан класс Преподаватель:", self.name)
    def info(self):
        Human.info(self)
        print('Кафедра:"{0}"Должность:"{1}"'.format(self.kafedra,
self.dolgnost))
#------------------------------------------------------------
class Student(Human):
    def __init__(self,name,fakultet,gruppa):
        Human.__init__ (self,name)
        self.fakultet=fakultet #Название факультета
        self.gruppa=gruppa #Номер группы
        print("Создан класс Студент:",self.name)
    def info(self):
        Human.info(self)
        print('Факультет:"{0}"Группа:"{1}"'.format(self.fakultet,self.gruppa))
#------------------------------------------------------------
pr=Prepod("Сергей Гуриков","Информатика", "Доцент")
pr.info()
print()
```

```
st=Student("Макс Рыжиков", "ОТФ-1", "БАП1601")
st.info()
```

4.2 Множественное наследование

В определении класса в круглых скобках можно указать сразу несколько базовых классов через запятую. Рассмотрим множественное наследование на примере.

```
class Class1: # Базовый класс для класса Class2
    def func1 (self):
        print ("Метод func1() класса Class")
class Class2(Class1): # Класс Class2 наследует  класс Class1
    def func2(self):
        print("Метод func2() класса Class2")
class Class3 (Class1) : # Класс Class3 наследует класс Classl
    def func1(self) :
        print ("Метод func1() класса Class3")
    def func2(self):
        print ("Метод func2() класса Class3")
    def func3(self):
        print ("Метод func3() класса Class3")
    def func4(self):
        print ("Метод func4() класса Class3")
class Class4 (Class2, Class3): # Множественное наследование
    def func4(self):
        print ("Метод func4() класса Class4")
c = Class4() #  Создаем экземпляр класса  Class4
c.func1()    #  Выведет: Метод func1() класса Class3
c.func2()    #  Выведет: Метод func2() класса Class2
c.func3()    #  Выведет: Метод func3() класса Class3
c.func4()    #  Выведет: Метод func4() класса Class4
```

Результат работы приложения:

```
Метод func1() класса Class3
Метод func2() класса Class2
Метод func3() класса Class3
Метод func4() класса Class4
```

Метод **func1()** определен в двух классах: **Class1** и **Class3**. Так как вначале просматриваются все базовые классы, непосредственно указанные в определении текущего класса, метод **func1()** будет найден в классе **Class3** (поскольку он указан в числе базовых классов в определении **Class4**), а не в классе **Class1**.

Метод **func2()** также определен в двух классах: **Class2** и **Class3**. Так как класс **Class2** стоит первым в списке базовых классов, то метод будет найден именно в нем. Чтобы наследовать метод из класса **Class3**, следует указать это явным образом. Переделаем определение класса **Class3** из предыдущего примера и наследуем метод **func2()** из класса **Class3**:

```
class Class4 (Class2, Class3): # Множественное наследование
    #  Наследуем func2() из класса Class3, а не из класса Class2
    func2 = Class3.func2
    def func4(self):
        print ("Метод func4() класса Class4")
c = Class4() #  Создаем экземпляр класса  Class4
c.func1()   #  Выведет: Метод func1() класса Class3
c.func2()   #  Выведет: Метод func2() класса Class2
c.func3()   #  Выведет: Метод func3() класса Class3
c.func4()   #  Выведет: Метод func4() класса Class4
```

Результат работы приложения:

```
Метод func1() класса Class3
Метод func2() класса Class3
Метод func3() класса Class3
Метод func4() класса Class4
```

Вернемся к первому примеру. Метод **func3()** определен только в классе **Class3**, поэтому метод наследуется от этого класса. Метод **func4()**, определенный в классе **Class3**, переопределяется в производном классе.

Если же искомый метод найден в производном классе, то вся иерархия наследования просматриваться не будет.

Для получения перечня базовых классов можно воспользоваться атрибутом __bases__.

В качестве значения атрибут возвращает кортеж. В качестве примера выведем базовые классы для всех классов из предыдущего примера:

```
print(Class1.__bases__)
print(Class2.__bases__)
print(Class3.__bases__)
print(Class4.__bases__)
```

Результат работы приложения:

```
(,)
(<class '__main__.Class1'>,)
(<class '__main__.Class1'>,)
(<class '__main__.Class2'>, <class '__main__.Class3'>)
```

Приведенный ниже пример проиллюстрирует порядок поиска идентификаторов при сложной иерархии множественного наследования:

```
class Class1: x = 10
class Class2(Class1): pass
class Class3(Class2): pass
class Class4(Class3): pass
class Class5(Class2): pass
class Class6(Class5): pass
class Class7(Class4, Class6): pass
c = Class7()
print(c.x)
print(Class7.__mro__)
```

Последовательность поиска атрибута **x** будет такой:

Class7 -> Class4 -> Class3 - > Class6 -> Class5 -> Class2 -> Class1

Получить всю цепочку наследования позволяет атрибут **__mro__**:

```
print(Class7.__mro__)
```

Результат выполнения:

```
10
(<class '__main__.Class7'>, <class '__main__.Class4'>, <class '__main__.Class3'>,
<class '__main__.Class6'>, <class '__main__.Class5'>, <class '__main__.Class2'>,
<class '__main__.Class1'>, <class 'object'>)
```

4.3 Примеси и их использование

Множественное наследование, поддерживаемое **Python**, позволяет реализовать интересный способ расширения функциональности классов с помощью так называемых **примесей (mixins)**. **Примесь** — это класс, включающий какие-либо атрибуты и методы, которые необходимо добавить к другим классам. Объявляются они точно так же, как и обычные классы:

```
class Mixin:              # Определяем сам класс-примесь
    attr = 0              # Определяем атрибут примеси
    def mixin_method(self): # Определяем метод примеси
        print("Метод примеси")
```

Теперь объявим два класса, добавим к их функциональности ту, что определена в классе-примеси **Mixin**, и проверим ее в действии:

```
class Class1 (Mixin):
    def method1(self):
        print("Метод класса Class1")
class Class2 (Class1, Mixin):
    def method2(self):
        print("Метод класса Class2")
c1 = Class1()
c1.method1()
c1.mixin_method()       # Classl поддерживает метод примеси
c2 = Class2()
c2.method1()
c2.method2()
c2.mixin_method()       # Class2 также поддерживает метод примеси
```

Результат выполнения приложения:

```
Метод класса Class1
Метод примеси
Метод класса Class1
Метод класса Class2
Метод примеси
```

Примеси активно применяются в различных дополнительных библиотеках - в частности, в популярной библиотеке **Web**-программирования **Django**.

4.4 Пример решения задачи на языке программирования Python

Задача. Создайте класс **ПЕРСОНА** с методами, позволяющими вывести на экран информацию о персоне, а также определить ее возраст (в текущем году). Создайте дочерние классы: **АБИТУРИЕНТ** (фамилия, дата рождения, факультет), **СТУДЕНТ** (фамилия, дата рождения, факультет, курс), **ПРЕПОДАВАТЕЛЬ** (фамилия, дата рождения, факультет, должность, стаж), со своими методами вывода информации на экран и определения возраста. Создайте список из **n** персон, выведите полную информацию из базы на экран, а также организуйте поиск персон, чей возраст попадает в заданный диапазон.

Комментарий. В родительском классе **Persona()** определим, в соответствии с условием задачи, метод **vozrast(),** служащий для определения возраста и метод **info(),** позволяющий вывести информацию о персоне. Далее создаем три дочерних класса: **Abiturient(Persona), Student(Persona), Prepodavatel(Persona),** основанные на классе **Persona().** Соответственно, все дочерние классы будут наследовать методы родительского класса. Чтобы вызвать конструктор базового класса, можно использовать функцию Python – **super().** Заметим, что при использовании функции **super()** можно не передавать в явном виде параметр **self.** Таким образом, вместо инструкции **Persona.__init__(self, name, date, tdate)** в дочернем классе **Student(Persona)** используется инструкция **super().__init__(name,date, tdate).** Подобная ситуация повторяется и в других классах.

В основной части программы мы организуем ввод количества персон и запрашиваем текущий год. В соответствии с количеством персон начинаем осуществлять запрос данных в зависимости от введенной пользователем категории персоны: абитуриент, студент, преподаватель. Создав экземпляры объектов **abitur, stud** и **prepod** классов **Abiturient, Student, Prepodavatel,** передаем им соответствующие значения.

В конце программы осуществляем поиск персон, чей возраст попадает в заданный диапазон. Для этого у нас уже сформирован список персон, хранящийся в соответствующем списке **itogspisok.** Получив критерии **diapazon1** и **diapazon2,** программа благополучно отфильтрует нужные сведения.

Полный код программы представлен ниже.

```
class Persona():
    vozr=0
    def __init__(self,name,date,tdate):
        self.name=name
        self.date=date
        self.tdate=tdate
    def vozrast(self):
         self.vozr=self.tdate-self.date
```

```
    def info(self):
        print("\n Созданн класс Персона")
        print('Фамилия Имя:" {0} "Возраст:" {1}"'.format(self.name,self.vozr))
#создание дочерних классов
class Abiturient(Persona):
    def __init__(self,name,date,tdate,fakultet):
        super().__init__(name,date, tdate)
        self.fakultet=fakultet
    def info(self):
        Persona.vozrast(self)
        Persona.info(self)
        print("Создан класс Абитруиент")
        print('Факультет: "{0}"'.format(self.fakultet))
class Student(Persona):
    def __init__(self,name,date,tdate,fakultet,kurs):
        super().__init__(name,date,tdate)
        self.fakultet=fakultet
        self.kurs=kurs
    def info(self):
        Persona.vozrast(self)
        Persona.info(self)
        print("Создан класс Студент")
        print('Факультет:"{0}" Курс:"{1}"'.format(self.fakultet,self.kurs))
class Prepodavatel(Persona):
    def __init__(self,name,date,tdate,fakultet,dolgnost,staj):
        super().__init__(name,date,tdate)
        self.fakultet=fakultet
        self.dolgnost=dolgnost
        self.staj=staj
    def info(self):
        Persona.vozrast(self)
        Persona.info(self)
        print("Создан класс Преподаватель")
        print('Факультет: "{0}" Должность: "{1}"
Стаж:"{2}"'.format(self.fakultet,self.dolgnost,self.staj))
#----------Основная часть программы----------
itogspisok=[]
n=int(input("Введите количество персон: "))
tdate1=int(input("Введите текущий год: "))
for i in range (n):
    persona=input ("Кто вы? 1)Абитуриент, 2)Студент, 3)Преподаватель)")
    name1=input("Введите вашу фамилию и имя: ")
    date1=int(input("Введите год рождения:"))
    spisok=[name1,date1]
    itogspisok.append(spisok)
```

```
    if persona=="1":
        fakultet1=input("Введите факультет:")
        abitur=Abiturient(name1,date1,tdate1,fakultet1)
        abitur.info()
        print()
    if persona=="2":
        fakultet1=input("Введите факультет:")
        kurs1=input("Введите курс:")
        stud=Student(name1,date1,tdate1,fakultet1,kurs1)
        stud.info()
        print()
    if persona=="3":
        fakultet1=input("Введите факультет:")
        dolgnost1=input("Введите должность:")
        staj1=input("Введите стажработы:")
        prepod=Prepodavatel(name1,date1,tdate1,fakultet1,dolgnost1,staj1)
        prepod.info()
        print()
#----------Организация поиски персон, чей возраст попадает в заданный
#диапазон----------
print("Далее будут отобраны персоны, чей год рождения попадает в
заданный промежуток")
diapazon1=int(input("Введите начало промежутка:"))
diapazon2=int(input("Введите конец промежутка:"))
print("\n Персоны, удовлетворяющие заданному условию:")
for i in range (len(itogspisok)):
    if (itogspisok[i][1]>=diapazon1) and (itogspisok[i][1]<=diapazon2):
        print("Фамилия Имя: ", itogspisok[i][0], "Год
рождения:",itogspisok[i][1])
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте класс **ФИГУРА** с методами вычисления площади и периметра, а также методом, выводящим информацию о фигуре на экран. Создайте дочерние классы **ПРЯМОУГОЛЬНИК, КРУГ, ТРЕУГОЛЬНИК** со своими методами вычисления площади и периметра. Создайте список **n** фигур и выведите полную информацию о фигурах на экран.

2 Создайте класс **ИЗДАНИЕ** с методом, позволяющим вывести на экран информацию об издании, а также определить, является ли данное издание искомым. Создайте дочерние классы **КНИГА** (название, фамилия автора, год издания, издательство), **СТАТЬЯ** (название, фамилия автора, название журнала,

его номер и год издания), **ЭЛЕКТРОННЫЙ РЕСУРС** (название, фамилия автора, ссылка, аннотация) со своими методами вывода информации на экран. Создайте список из **n** изданий, выведите полную информацию из списка, а также организуйте поиск изданий по фамилии автора.

3 Создайте класс **ТРЕУГОЛЬНИК,** заданный длинами двух сторон и угла между ними, с методами вычисления площади и периметра треугольника, а также методом, выводящим информацию о фигуре на экран. Создайте дочерние классы **ПРЯМОУГОЛЬНЫЙ, РАВНОБЕДРЕННЫЙ, РАВНОСТОРОННИЙ** со своими методами вычисления площади и периметра. Создайте список **n** треугольников и выведите полную информацию о треугольниках на экран.

4 Создайте класс **ТРАНСПОРТ** с методами, позволяющими вывести на экран информацию о транспортном средстве, а также определить грузоподъемность транспортного средства. Создайте дочерние классы **АВТОМОБИЛЬ** (марка, номер, скорость, грузоподъемность), **МОТОЦИКЛ** (марка, номер, скорость, грузоподъемность, наличие коляски, при этом если коляска отсутствует, то грузоподъемность равна нулю), **ГРУЗОВИК** (марка, номер, скорость, грузоподъемность, наличие прицепа, при этом если есть прицеп, то грузоподъемность увеличивается в два раза) со своими методами вывода информации на экран и определения грузоподъемности. Создайте список из **n** машин, выведите полную информацию на экран, а также организуйте поиск машин, удовлетворяющих требованиям грузоподъемности.

5 Создайте класс **ТОВАР** с методами, позволяющими вывести на экран информацию о товаре, а также определить, может ли приобрести товар покупатель, имеющий заданную сумму денег. Создайте дочерние классы **ПРОДУКТ** (название, цена, дата производства, срок годности), **ПАРТИЯ** (название, цена за штуку, количество штук, дата производства, срок годности), **ТЕЛЕФОН** (название, цена) со своими методами вывода информации на экран и определения соответствия заданной цене. Создайте список из **n** товаров, выведите полную информацию из базы на экран, а также организуйте поиск товара, который может приобрести покупатель, имеющий заданную сумму денег.

6 Создайте класс **ТОВАР** с методами, позволяющими вывести на экран информацию о товаре, а также определить, предназначен ли он для заданного возраста потребителя. Создайте дочерние классы **ИГРУШКА** (название, цена, производитель, материал, возраст, на который рассчитана), **КНИГА** (название, автор, цена, издательство, возраст, на который рассчитана), **СПОРТИНВЕНТАРЬ** (название, цена, производитель, возраст, на который рассчитан) со своими методами вывода информации на экран и определения соответствия возрасту потребителя. Создайте список из **n** товаров, выведите полную информацию из базы на экран, а также организуйте поиск товаров для потребителя в заданном возрастном диапазоне.

7 Создайте класс **ТЕЛЕФОННЫЙ_СПРАВОЧНИК** с методами, позволяющими вывести на экран информацию о записях в телефонном справочнике, а также определить соответствие записи критерию поиска. Создайте дочерние классы **ПЕРСОНА** (фамилия, адрес, номер телефона), **ОРГАНИЗАЦИЯ** (название, адрес, телефон, факс, контактное лицо), **ДРУГ**

(фамилия, адрес, номер телефона, дата рождения) со своими методами вывода информации на экран и определения соответствия заданной фамилии. Создайте список из **n** записей, выведите полную информацию из базы на экран, а также организуйте поиск в базе по фамилии.

8 Создайте класс **КЛИЕНТ** с методами, позволяющими вывести на экран информацию о клиентах банка, а также определить соответствие клиента критерию поиска. Создайте дочерние классы **ВКЛАДЧИК** (фамилия, дата открытия вклада, размер вклада, процент по вкладу), **КРЕДИТОР** (фамилия, дата выдачи кредита, размер кредита, процент по кредиту, остаток долга), **ОРГАНИЗАЦИЯ** (название, дата открытия счета, номер счета, сумма на счету) со своими методами вывода информации на экран и определения соответствия дате (открытия вклада, выдаче кредита, открытия счета). Создайте список из **n** клиентов, выведите полную информацию из базы на экран, а также организуйте поиск клиентов, начавших сотрудничать с банком в заданную дату.

9 Создайте класс **ПРОГРАММНОЕ_ОБЕСПЕЧЕНИЕ** с методами, позволяющими вывести на экран информацию о программном обеспечении, а также определить соответствие возможности использования (на текущую дату). Создайте дочерние классы **СВОБОДНОЕ** (название, производитель), **УСЛОВНО_БЕСПЛАТНОЕ** (название, производитель, дата установки, срок бесплатного использования), **КОММЕРЧЕСКОЕ** (название, производитель, цена, дата установки, срок использования) со своими методами вывода информации на экран и определения возможности использования на текущую дату. Создайте список из **n** видов программного обеспечения, выведите полную информацию из базы на экран, а также организуйте поиск программного обеспечения, которое допустимо использовать на текущую дату.

10 Создайте класс **ТРАНСПОРТ** с методами, позволяющими вывести на экран информацию о транспортном средстве, а также определить, находится ли транспортное средство в пределах заданных координат. Создайте дочерние классы **САМОЛЕТ** (марка, максимальные скорость и высота, количество пассажиров, координаты), **АВТОМОБИЛЬ** (марка, номер, год выпуска, координаты), **КОРАБЛЬ** (название, координаты, скорость, количество пассажиров, порт приписки) со своими методами вывода информации на экран и определения присутствия транспортного средства в пределах заданных координат. Создайте список из **n** транспортных средств, выведите полную информацию из базы на экран, а также организуйте поиск транспортных средств, которые сейчас находятся в пределах заданных координат.

11 Создайте класс **ИГРУШКА** с методами, позволяющими вывести на экран информацию о товаре, а также определить соответствие игрушки критерию поиска. Создайте дочерние классы **КУБИК** (цвет, цена, материал, размер ребра), **МЯЧ** (цена, цвет, диаметр, материал), **МАШИНКА** (название, цена, производитель, цвет) со своими методами вывода информации на экран и определения соответствия заданному цвету. Создайте список из **n** игрушек, выведите полную информацию из базы на экран, а также организуйте поиск игрушек заданного цвета.

12 Создайте класс **ТЕЛО** с методами вычисления площади поверхности и объема, а также методом, выводящим информацию о фигуре на экран. Создайте дочерние классы **ПАРАЛЛЕЛЕПИПЕД, ШАР, ПИРАМИДА** со своими методами вычисления площади и объема. Создайте список **n** фигур и выведите полную информацию о фигурах на экран.

13 Создайте класс **УРАВНЕНИЕ** с методами вычисления корня уравнения и вывода результата на экран. Создайте дочерние классы **ЛИНЕЙНОЕ, КВАДРАТНОЕ** со своими методами вычисления корней и вывода на экран. Создайте список **n** уравнений и выведите полную информацию об уравнениях на экран.

14 Создайте класс **ВАЛЮТА** с методами перевода денежной суммы в рубли и вывода на экран. Создайте дочерние классы **ДОЛЛАР, ЕВРО** со своими методами перевода и вывода на экран. Создайте список **n** валютных денежных сумм и выведите полную информацию о них на экран.

15 Создайте класс **ПРОГРЕССИЯ** с методами вычисления j-го элемента прогрессии, ее суммы и методом, выводящим сумму на экран. Создайте дочерние классы: **АРИФМЕТИЧЕСКАЯ, ГЕОМЕТРИЧЕСКАЯ** со своими методами вычисления. Создайте список **n** прогрессий и выведите сумму каждой из них экран.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Охарактеризуйте наследование в языке программирования Python.
7.2 Охарактеризуйте множественное наследование в языке программирования Python.
7.3 Раскройте особенности одного из основных принципов ООП – наследования. Приведите синтаксис создания производного класса в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.